Practice · Whole programs 2

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What the ticket program does

Read the whole program before you answer.

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Age: ");
    scanf("%d", &age);
    if (age >= 18) {
        printf("Adult ticket\n");
    } else {
        printf("Student ticket\n");
    }
    return 0;
}
```

What does it do?

- It prints both ticket types, one after the other.
- **Right answer:** It asks for an age and prints one of two ticket types, depending on the age.
- It prints `Adult ticket` and then stops.
- It asks for an age and prints it back.

Feedback: Ask, read, decide, print. `scanf` reads the age. The `if` looks at it: 18 or more, the first block runs; less, the `else` block runs. Only one of the two `printf` lines ever runs. This is slide 12.

(mc) L1 · Which line decides

Same program:

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Age: ");
    scanf("%d", &age);
    if (age >= 18) {
        printf("Adult ticket\n");
    } else {
        printf("Student ticket\n");
    }
    return 0;
}
```

Which line chooses between the two tickets?

- Line 4, `printf("Age: ");`
- Line 5, `scanf("%d", &age);`

- **Right answer:** Line 6, `if (age >= 18) {`
- Line 8, `} else {`

Feedback: The `if` line holds the condition. Everything else either gets the age or prints a result. `else` on its own decides nothing; it is only the other door of the same `if`.

(mc) L1 · The shape of every program this week

Most programs this week have the same steps in the same order. Which order?

- Decide, ask, read, print.
- **Right answer:** Ask, read, decide, print.
- Read, print, decide, ask.
- Ask, decide, read, print.

Feedback: Look at the ticket program: `printf("Age: ")` asks, `scanf` reads, `if` decides, `printf` prints. Last week it was ask, read, calculate, print. This week the "calculate" step often becomes a decision, or sits next to one, as in the BMI program.

(mc) L1 · Why double in the BMI program

```
#include <stdio.h>
int main(void) {
    double weight, height, bmi;
    printf("Weight and height: ");
    scanf("%lf %lf", &weight, &height);
    bmi = weight / (height * height);
    printf("BMI: %.2f\n", bmi);
    if (bmi < 18.5) {
        printf("Underweight\n");
    } else if (bmi < 25.0) {
        printf("Normal\n");
    } else if (bmi < 30.0) {
        printf("Overweight\n");
    } else {
        printf("Obese\n");
    }
    return 0;
}
```

Why are `weight`, `height` and `bmi` all `double`?

- `if` only works with `double`.
- **Right answer:** A height like 1.75 and a BMI like 22.86 have decimal parts, and `int` would throw them away.
- `scanf` cannot read an `int`.
- No reason. `int` would give the same result.

Feedback: Height is 1.75, not 1 or 2. BMI is 22.86, not 22. Any value with a decimal part needs `double`. With `int height`, 1.75 would become 1 and every BMI would be wrong.

(mc) L1 · Why the second condition is so short

In the BMI program the second branch is `else if (bmi < 25.0)`, not `else if (bmi >= 18.5 && bmi < 25.0)`. Why is the short form enough?

- It is not enough. The short form is a bug.
- `&&` does not work with `double`.
- **Right answer:** The program only reaches this branch when `bmi < 18.5` was already false, so `bmi >= 18.5` is already known.
- `else if` ignores the first half of a condition.

Feedback: A chain is checked top to bottom, and each branch is tried only if the ones above were false. If `bmi < 18.5` failed, `bmi` is at least 18.5. So the second branch only needs to check the top of its range. Slide 18's "answer idea".

(mc) L1 · What the bill program calculates

```
#include <stdio.h>
int main(void) {
    double kwh, bill;
    printf("kWh: ");
    scanf("%lf", &kwh);
    if (kwh <= 100) {
        bill = kwh * 0.10;
    } else if (kwh <= 200) {
        bill = 100 * 0.10 + (kwh - 100) * 0.15;
    } else {
        bill = 100 * 0.10 + 100 * 0.15 + (kwh - 200) * 0.20;
    }
    printf("Bill = %.2f\n", bill);
    return 0;
}
```

What is the rule this program applies?

- Every kWh costs 0.20.
- The price depends on the day of the week.
- **Right answer:** The first 100 kWh cost 0.10 each, the next 100 cost 0.15 each, and everything above 200 costs 0.20 each.
- Bills above 200 are free.

Feedback: Three price bands, so three branches. Up to 100: all at 0.10. Up to 200: the first 100 at 0.10, the rest at 0.15. Above 200: the first 100 at 0.10, the next 100 at 0.15, the rest at 0.20. Tiered pricing is slide 19.

(mc) L1 · Which line waits in the menu program

```
#include <stdio.h>
int main(void) {
    char option;
    printf("A/B/C: ");
    scanf(" %c", &option);
    switch (option) {
        case 'A':
            printf("Add item\n");
            break;
```

```
        case 'B':
            printf("View cart\n");
            break;
        default:
            printf("Unknown option\n");
    }
    return 0;
}
```

Which line makes the program stop and wait for you to type?

- Line 4, `printf("A/B/C: ");`
- **Right answer:** Line 5, `scanf(" %c", &option);`
- Line 6, `switch (option) {`
- Line 7, `case 'A':`

Feedback: `printf` only shows the menu and moves on. `scanf` waits for a character. `switch` then jumps to the matching `case`. Reading a `char` needs `%c` and the space before it (slide 16).

(mc) L1 · What the even-or-odd program checks

```
#include <stdio.h>
int main(void) {
    int n;
    printf("Number: ");
    scanf("%d", &n);
    if (n % 2 == 0) {
        printf("Even\n");
    } else {
        printf("Odd\n");
    }
    return 0;
}
```

What does the condition `n % 2 == 0` ask?

- Is `n` equal to 2?
- Is `n` bigger than 2?
- **Right answer:** Is the remainder after dividing `n` by 2 equal to 0?
- Does `n` have two digits?

Feedback: `%` gives the remainder. An even number leaves 0 when divided by 2, an odd number leaves 1. So `n % 2 == 0` is true exactly for even numbers. This is task 2 on slide 21.

(mc) L1 · Overtime, which branch

```
#include <stdio.h>
int main(void) {
    double hours, rate, pay;
    printf("Hours and rate: ");
    scanf("%lf %lf", &hours, &rate);
    if (hours <= 40) {
        pay = hours * rate;
```

```
    } else {
        pay = 40 * rate + (hours - 40) * rate * 1.5;
    }
    printf("Pay: %.2f\n", pay);
    return 0;
}
```

What does the `else` branch calculate?

- Every hour at one and a half times the rate.
- Forty hours at the rate, and nothing for the rest.
- **Right answer:** Forty hours at the normal rate, plus the hours above forty at one and a half times the rate.
- The hours above forty at the normal rate.

Feedback: Overtime is only the part above 40. So: `40 * rate` for the normal hours, plus `(hours - 40) * rate * 1.5` for the extra ones. This is task 5 on slide 21.

(mc) L2 · Ticket for 18

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Age: ");
    scanf("%d", &age);
    if (age >= 18) {
        printf("Adult ticket\n");
    } else {
        printf("Student ticket\n");
    }
    return 0;
}
```

You type `18`. What is printed after `Age: ` ?

- **Right answer:** `Adult ticket`
- `Student ticket`
- Both lines.
- Nothing.

Feedback: `18 >= 18` is true, because `>=` means "greater **or equal**". So the `if` block runs. Type `17` and you get `Student ticket`. The border case is always worth testing.

(mc) L2 · Grade for 85

```
int score;
printf("Score: ");
scanf("%d", &score);
if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else if (score >= 70) {
    printf("C\n");
```

```
} else {
    printf("Needs improvement\n");
}
```

You type `85`. What is printed after `Score:` ?

- `A`
- **Right answer:** `B`
- `B` and `C`
- `C`

Feedback: `85 >= 90` is false, so move on. `85 >= 80` is true, so `B` is printed and the rest of the chain is skipped. With `95` you get `A`, with `69` you get `Needs improvement`.

(mc) L2 · Menu with a lower-case letter

```
char option;
printf("A/B/C: ");
scanf(" %c", &option);
switch (option) {
    case 'A':
        printf("Add item\n");
        break;
    case 'B':
        printf("View cart\n");
        break;
    default:
        printf("Unknown option\n");
}
```

You type a small `a`. What is printed after `A/B/C:` ?

- `Add item`
- **Right answer:** `Unknown option`
- Nothing.
- The program crashes.

Feedback: `'a'` and `'A'` are two different characters to C. No `case` matches a small `a`, so `default` runs. If you want both to work, you need a second case: `case 'a':` above `case 'A':` with no `break` between them.

(mc) L2 · BMI for 70 kg and 1.75 m

Use the BMI program from slide 18 (BMI = weight / (height × height); under 18.5 Underweight, under 25 Normal, under 30 Overweight, else Obese). It also prints the BMI with `%.2f` first. You type `70 1.75`. What are the two lines printed?

- `BMI: 40.00` and `Obese`
- `BMI: 22.86` and `Overweight`
- **Right answer:** `BMI: 22.86` and `Normal`
- `BMI: 22.00` and `Normal`

Feedback: 70 / (1.75 × 1.75) = 70 / 3.0625 = 22.857..., shown as `22.86`. Then the chain: `22.86 < 18.5` is false, `22.86 < 25.0` is true, so `Normal`.

(mc) L2 · Bill for 150 kWh

Use the electricity bill program from slide 19: first 100 kWh at 0.10, next 100 at 0.15, above 200 at 0.20. You type `150`. What is printed?

- `Bill = 15.00`
- `Bill = 22.50`
- **Right answer:** `Bill = 17.50`
- `Bill = 7.50`

Feedback: 150 is more than 100 and at most 200, so the middle branch runs: 100 × 0.10 = 10.00 for the first hundred, plus (150 − 100) × 0.15 = 7.50 for the next fifty. Total 17.50. For 80 the bill is 8.00, for 250 it is 35.00.

(mc) L2 · The tracing exercise

```
int x = 5;
int y = 12;
if (x * 2 > y) {
    printf("A");
} else if (y % x == 2) {
    printf("B");
} else {
    printf("C");
}
printf("D");
```

What exactly is printed?

- `AD`
- **Right answer:** `BD`
- `CD`
- `B`

Feedback: `x * 2` is 10, and `10 > 12` is false. `y % x` is `12 % 5`, which is 2, and `2 == 2` is true, so `B` is printed. The last `printf("D")` is outside the chain and always runs. So `BD`, on one line, because there is no `\n`. Slide 20.

(mc) L2 · The tracing exercise with x = 7

Same program, but now `x` is `7` and `y` is still `12`. What is printed?

- **Right answer:** `AD`
- `BD`
- `CD`
- `AB`

Feedback: `7 * 2` is 14, and `14 > 12` is true, so `A` is printed and the other branches are skipped. Then `D`. Change one number and the path through the program changes. That is what tracing shows you.

(mc) L2 · Find the bug: the ticket that is always adult

```
#include <stdio.h>
int main(void) {
    int age;
```

```
    printf("Age: ");
    scanf("%d", &age);
    if (age = 18) {
        printf("Adult ticket\n");
    } else {
        printf("Student ticket\n");
    }
    return 0;
}
```

It compiles with a warning, `suggest parentheses around assignment used as truth value`, and prints `Adult ticket` for every age, even `5`. What is the bug?

- `int age;` should be `double age;`.
- **Right answer:** The condition uses `=` instead of `==`, so it puts 18 into `age` and is always true.
- The `else` should come before the `if`.
- `%d` should be `%lf`.

Feedback: `age = 18` is not a question. It stores 18 in `age`, and the value of that is 18, which is not zero, so C treats it as true. The fix is `age == 18`, or, for the real ticket program, `age >= 18`. Watch for this warning: it almost always means a missing `=`.

(mc) L2 · Find the bug: the menu that prints two things

```
switch (option) {
    case 'A':
        printf("Add item\n");
    case 'B':
        printf("View cart\n");
        break;
    default:
        printf("Unknown option\n");
}
```

You type `A` and see `Add item` and then `View cart`. What is the bug?

- `case 'A':` should be `case "A":`.
- `default` should come first.
- **Right answer:** There is no `break;` after `printf("Add item\n");`, so the program falls through into case B.
- `switch` cannot have more than one `case`.

Feedback: Each `case` needs a `break;` at its end, or C carries on into the next case. Add `break;` after `Add item` and typing `A` prints one line only.

(mc) L2 · Find the bug: the missing &

```
#include <stdio.h>
int main(void) {
    int score;
    printf("Score: ");
    scanf("%d", score);
    if (score >= 50) {
```

```
        printf("Passed\n");
    }
    printf("Done\n");
    return 0;
}
```

`gcc -Wall` prints a warning on line 5: `format '%d' expects argument of type 'int *', but argument 2 has type 'int'`. You type `60` and see only `Done`. What is wrong?

- `score` should be a `double`.
- `if` needs an `else`.
- **Right answer:** The `scanf` line is missing the `&`, so the number you typed never reaches `score`.
- `printf("Done\n")` should be inside the braces.

Feedback: Without `&`, `scanf` does not know where the box is. The typed 60 goes nowhere and `score` keeps its rubbish, so the condition is false, or the program crashes. Write `scanf("%d", &score);`. The warning tells you the line.

(fib) L3 · Complete the ticket: the else branch

Complete the keyword so the second ticket prints only when the age is under 18.

```
#include <stdio.h>
int main(void) {
    int age;
    printf("Age: ");
    scanf("%d", &age);
    if (age >= 18) {
        printf("Adult ticket\n");
    } [else] {
        printf("Student ticket\n");
    }
    return 0;
}
```

Feedback: `else` is the other door of the `if`: it runs when the condition is false. Type `18` and you get the adult ticket, `17` the student one.

(fib) L3 · Complete the grade chain: the second branch

Complete the two keywords that start the branch for a B. Type them with one space between them.

```
int score;
printf("Score: ");
scanf("%d", &score);
if (score >= 90) {
    printf("A\n");
} [else if] (score >= 80) {
    printf("B\n");
} else {
    printf("C\n");
}
```

Feedback: `else if` adds another branch that is only tried when the ones above it were false. Two words.

(fib) L3 · Complete the grade chain: the threshold for C

Complete the number so that a C is given for 70 to 79.

```
if (score >= 90) {
    printf("A\n");
} else if (score >= 80) {
    printf("B\n");
} else if (score >= [70]) {
    printf("C\n");
} else {
    printf("Needs improvement\n");
}
```

Feedback: The branches above already catch 80 and up, so this one only needs its own bottom: 70. A 79 fails the `>= 80` test and passes `>= 70`, so it gets a C.

(fib) L3 · Complete the menu: the break

Complete the line that stops case A from running into case B. Type it exactly, with the semicolon.

```
switch (option) {
    case 'A':
        printf("Add item\n");
        [break;]
    case 'B':
        printf("View cart\n");
        break;
    default:
        printf("Unknown option\n");
}
```

Feedback: `break;` leaves the `switch`. Without it, `A` would print both `Add item` and `View cart`.

(fib) L3 · Complete the BMI program: the read line

Complete the two holes so both weight and height are read as `double`. Type the two holes with one space between them, like `%d %d`.

```
double weight, height, bmi;
printf("Weight and height: ");
scanf("[%lf %lf]", &weight, &height);
bmi = weight / (height * height);
printf("BMI: %.2f\n", bmi);
```

Feedback: `%lf` is the `scanf` hole for a `double`, and two boxes need two holes. In `printf` a `double` is `%f` or `%.2f`; only `scanf` wants the `l`.

(fib) L3 · Complete the BMI program: the formula

Complete the calculation of BMI: weight divided by height squared. Type it with single spaces, exactly like `a / (b * b)`.

```
double weight, height, bmi;
printf("Weight and height: ");
scanf("%lf %lf", &weight, &height);
bmi = [weight / (height * height)|weight/(height*height)|weight / height /
height|weight/height/height];
printf("BMI: %.2f\n", bmi);
```

Feedback: Height squared is `height * height`, and the brackets make sure the whole square is the divisor. `weight / height / height` gives the same answer, because dividing by height twice is dividing by height squared.

(fib) L3 · Complete the even-or-odd program: the condition

Complete the condition so the first block runs for even numbers. Type it with single spaces, exactly like `n % 2 == 0`.

```
int n;
printf("Number: ");
scanf("%d", &n);
if ([n % 2 == 0|n%2==0]) {
    printf("Even\n");
} else {
    printf("Odd\n");
}
```

Feedback: An even number leaves remainder 0 when divided by 2. `n % 2` is that remainder, and `== 0` asks whether it is zero. Two equals signs.

(fib) L3 · Complete the entry check: the AND

Complete the condition so a person is allowed in only when they are 18 or older **and** have a card. Type only the symbol.

```
int age, hasCard;
printf("Age and card (1 or 0): ");
scanf("%d %d", &age, &hasCard);
if (age >= 18 [&&] hasCard) {
    printf("Allowed\n");
} else {
    printf("Not allowed\n");
}
```

Feedback: `&&` is AND. Both `age >= 18` and `hasCard` must be true. With `||` a five-year-old with a card would be let in.

(fib) L3 · Complete the average: the cast

Complete the cast so the average keeps its decimals. Type only what goes in the brackets, like `(double)`.

```
int total = 7;
```

```
int people = 2;
double each = [(double)|( double )|(double )|( double)]total / people;
printf("Each: %.2f\n", each);
```

Feedback: `(double)total` turns 7 into 7.0 before the division, so `7.0 / 2` is 3.5 and the line prints `Each: 3.50`. Without the cast the division is `int / int`, 3, and the line prints `Each: 3.00`.